

LegalEase

AI-Powered Legal Document Generator

Naan Mudhalvan Project

Sub-title: Pre-requisites

Project Prerequisite Documentation

This 20-page document presents the prerequisites required to develop, test, and deploy the LegalEase project.

1. Project Overview

Project Title

LegalEase: AI-Powered Legal Document Generator

Module / Sub-title

Pre-requisites

Overview

LegalEase is a proposed AI-powered web application that helps users create structured legal documents from guided inputs. The system is intended to simplify document preparation by providing templates, collecting relevant information, validating required fields, and generating a downloadable document.

Purpose of this Section

This section identifies the hardware, software, development tools, data requirements, security considerations, knowledge prerequisites, and deployment requirements needed before implementing the core functionalities of LegalEase.

2. Understanding the Project Requirements

Functional Understanding

The development team should understand the complete workflow: user registration, document-type selection, guided data entry, validation, document generation, preview, download, and history management.

Key Objectives

The application should be easy to use, modular, secure, maintainable, and capable of generating consistent document layouts. AI features should assist with drafting or structuring content while clearly communicating that generated material may require review by a qualified legal professional.

Expected Users

Potential users include students, individuals, small organizations, and administrative staff who need structured document drafts. The interface should avoid unnecessary technical complexity.

Requirement Preparation

Before coding, prepare a list of supported document types, fields required for each template, output formats, validation rules, user roles, and system constraints.

3. Hardware Pre-requisites

Development Computer

A modern laptop or desktop with a multi-core processor is recommended for development. A minimum of 8 GB RAM is suitable for basic web development; 16 GB or more provides better performance when running an IDE, database, browser, and local services together.

Storage

At least 10 GB of free development storage is recommended for source code, dependencies, database files, generated documents, test data, and project documentation.

Network

A stable internet connection is required for installing packages, accessing development documentation, using cloud services if selected, and testing deployed APIs.

Testing Devices

At least one desktop/laptop browser and one mobile device or mobile emulator should be available to test responsive behavior.

4. Software Pre-requisites

Operating System

The project can be developed on Windows, Linux, or macOS. The selected operating system should support the chosen programming language, database, IDE, and version-control tools.

Runtime and Language

Choose a supported backend language such as Python, JavaScript/TypeScript, or Java. Python with a web framework is one possible implementation approach.

Frontend

A modern frontend stack such as HTML, CSS, JavaScript, React, or another suitable framework can be used to create forms, dashboards, document previews, and responsive screens.

Database

PostgreSQL, MySQL, SQLite, or another suitable database can store user profiles, templates, document metadata, and audit information.

5. Development Environment

IDE / Editor

Install a development environment such as Visual Studio Code, IntelliJ IDEA, PyCharm, or another editor appropriate to the selected technology.

Package Management

Configure the relevant package manager, such as npm/pnpm for JavaScript projects or pip/Poetry for Python projects. Lock dependency versions where practical.

Environment Configuration

Use environment variables for database URLs, API keys, authentication secrets, and other configuration values. Never place production secrets directly in source code.

Project Structure

Prepare separate areas for frontend, backend, templates, services, database models, tests, configuration, static assets, and documentation.

6. Database Pre-requisites

Schema Planning

Design tables or collections for users, roles, document templates, template fields, generated documents, document versions, and audit records.

Data Integrity

Define primary keys, foreign keys, constraints, required fields, indexes, and suitable data types before implementing database operations.

Migrations

Use a migration mechanism so database structure changes are versioned and reproducible across development and deployment environments.

Backup Planning

If user-generated documents are stored, establish a backup and recovery strategy appropriate to the deployment environment.

7. AI and NLP Pre-requisites

AI Purpose

Define exactly where AI is used. Possible functions include extracting structured information from user input, suggesting document wording, summarizing provided information, or assisting with template completion.

Model Access

If an external AI API is used, create the required account, API credentials, usage limits, and environment configuration. If a local model is selected, confirm hardware and model-runtime requirements.

Prompt Design

Prepare controlled prompts and structured output formats so that AI responses can be validated before being inserted into a document.

Human Review

AI output should be treated as draft assistance rather than automatically verified legal advice. Provide an appropriate review step before final use.

8. Legal Document Template Pre-requisites

Template Collection

Prepare approved project templates for the initial document types. Each template should have clearly identified placeholders and required fields.

Field Definition

For each document, define field names, descriptions, data types, validation rules, optional fields, and dependencies between fields.

Formatting

Define page size, margins, headings, numbering, signature areas, tables, dates, and other formatting rules for generated documents.

Versioning

Maintain template versions so that changes can be tracked and previously generated documents remain associated with the correct template version.

9. User Interface Pre-requisites

Navigation

Plan screens such as login, dashboard, document selection, form entry, preview, download, and document history.

Forms

Use clear labels, helpful instructions, validation messages, required-field indicators, and logical grouping of related information.

Responsive Design

The interface should adapt to desktop, tablet, and mobile screen sizes.

Accessibility

Use readable typography, sufficient contrast, keyboard-friendly controls, descriptive labels, and accessible error messages.

10. Authentication and Authorization

Authentication

Implement secure registration and login using password hashing or a trusted identity provider. Session or token management must be handled securely.

Authorization

Define roles such as user and administrator if required. Restrict template administration, user management, and other privileged functions.

Password Security

Never store plain-text passwords. Apply strong password policies and secure password hashing.

Session Security

Use secure cookies or appropriately protected tokens, expiration policies, and logout mechanisms.

11. Security Pre-requisites

Input Validation

Validate all user input on the server side, even when client-side validation is present.

Injection Protection

Use parameterized database queries or ORM mechanisms and sanitize data before processing or rendering.

File Security

Validate uploaded files by type, size, and content where uploads are supported. Store generated files in controlled locations.

Secrets

Protect API keys, database credentials, signing keys, and other secrets through environment configuration or a secure secret-management system.

12. Document Generation Pre-requisites

Generation Library

Select a reliable document-generation approach for the required output, such as PDF or DOCX. The chosen library should support the project's formatting requirements.

Data Mapping

Create a mapping between form fields and template placeholders. Missing required values should prevent incomplete generation.

Preview

Provide a preview or review stage before download so users can check the generated content.

File Naming

Use predictable and safe file names without exposing sensitive information in file paths.

13. API Pre-requisites

API Design

Define endpoints for authentication, document types, templates, generation requests, history, downloads, and administration as required.

Validation

Document request and response formats and validate incoming JSON or form data.

Error Handling

Use consistent status codes and user-friendly error messages while avoiding disclosure of internal stack traces or secrets.

Documentation

Maintain API documentation using an appropriate format such as OpenAPI/Swagger if the project includes multiple services.

14. Testing Pre-requisites

Unit Testing

Prepare tests for validation rules, database operations, document field mapping, utility functions, and AI response parsing.

Integration Testing

Test communication among frontend, backend, database, AI service, and document-generation service.

User Acceptance Testing

Create realistic scenarios covering successful generation, missing information, invalid input, authentication errors, and document download.

Test Data

Use synthetic or non-sensitive sample data during development and testing.

15. Version Control and Collaboration

Git Repository

Create a Git repository for the project source code and documentation.

Branching

Use a simple branching strategy suitable for the team. Keep changes focused and reviewable.

Commits

Write meaningful commit messages and avoid committing generated secrets, database credentials, or unnecessary build files.

README

Prepare a README containing project purpose, prerequisites, setup steps, environment variables, database setup, run commands, and testing instructions.

16. Deployment Pre-requisites

Hosting

Select a suitable hosting environment for the frontend, backend, database, and generated documents.

Environment Variables

Configure production-specific values separately from development settings.

HTTPS

Use HTTPS for deployed services, especially where authentication and user-generated documents are involved.

Monitoring

Plan basic application logging, error monitoring, uptime checks, and resource monitoring.

17. Performance and Scalability

Expected Load

Estimate the number of users, document generations, storage requirements, and AI requests expected during demonstrations and future use.

Optimization

Avoid unnecessary database queries, compress suitable assets, paginate document history, and reuse templates efficiently.

AI Usage

Set request limits and monitor AI usage to control latency and cost.

Caching

Where appropriate, cache static templates or other non-sensitive data to improve response time.

18. Documentation and Project Deliverables

Technical Documentation

Prepare architecture diagrams, database schema, API descriptions, installation steps, testing notes, and deployment instructions.

User Documentation

Prepare a simple guide explaining how to register, select a document, enter information, review the result, and download the generated file.

Project Presentation

Prepare screenshots, workflow diagrams, module descriptions, and a demonstration dataset for the Naan Mudhalvan project presentation.

Maintenance

Document known limitations, future enhancements, dependency versions, and procedures for updating templates.

19. Pre-requisites Checklist

Checklist

- Development computer configured
- IDE installed
- Runtime and package manager installed
- Database installed/configured
- Git repository created
- Frontend and backend project structure prepared
- Initial document templates prepared
- AI service or local model selected
- Environment variables configured
- Authentication design completed
- Security requirements identified
- Document generation library selected
- Test cases drafted
- Deployment environment selected
- Project documentation started

Readiness Review

Before implementation begins, confirm that each item has an owner, a setup status, and a clear verification method. This reduces integration problems during development.

20. Final Pre-requisite Summary

Summary

LegalEase requires a prepared development environment, clearly defined legal-document templates, a database design, secure authentication, a suitable AI integration strategy, document-generation tooling, testing infrastructure, and deployment planning.

Implementation Readiness

Once these prerequisites are completed, the team can proceed to the core functionalities: user management, document selection, guided forms, AI-assisted processing, document generation, preview, download, and history.

Important Note

LegalEase is a software project for document-generation assistance. Generated documents should be reviewed for factual accuracy, completeness, jurisdictional requirements, and suitability by an appropriately qualified person before being used for legal purposes.

Conclusion

A well-defined prerequisite stage provides the technical foundation for building LegalEase in a structured and maintainable way. The project can then progress from setup to implementation, testing, demonstration, and deployment.